Master 2 Intelligence Artificielle
Cours : Systèmes Multi-Agents

Conception d'un système multi-agents de réponse automatique aux e-mails avec des LLM exécutés localement

Rapport de projet

Membres du groupe

Prénom NOM	**Numéro étudiant**
Valentin **[À COMPLÉTER : NOM]**	**[À COMPLÉTER : n° étudiant]**
Kyllian **[À COMPLÉTER : NOM]**	**[À COMPLÉTER : n° étudiant]**
Jade **[À COMPLÉTER : NOM]**	**[À COMPLÉTER : n° étudiant]**
Théolien BIERHOFF	**[À COMPLÉTER : n° étudiant]**

Un seul étudiant du groupe dépose le projet, conformément à la consigne : **[À COMPLÉTER : nom du déposant]**.

Dépôt : `https://github.com/Biethe/MAS_Valentin_Kyllian_Jade_Bierhoff`
(branche `merge-email-multiagent`)

13 novembre 2026

Table des matières

1 Problème étudié et objectifs **2**

1.1 Problème étudié . . . 2

1.2 Objectifs du système . . . 2

2 Organisation multi-agents retenue **2**

2.1 Niveau 1 : routage par catégorie (module `email-multiagent`) . . . 3

2.2 Niveau 2 : planificateur et flux de travail (module `assistant-core`) . . . 3

2.3 Pourquoi deux niveaux ? Justification et état de l'intégration . . . 3

3 Architecture globale **4**

3.1 Modules et dépendances . . . 4

3.2 Classes et interfaces importantes . . . 4

3.3 Diagramme de classes simplifié (ports et adaptateurs) . . . 5

3.4 Mécanismes de communication entre agents . . . 5

4 Spécialisation des agents et stratégie de routage **6**

4.1 Spécialisation : prompt système et contexte . . . 6

4.2 Stratégies de routage (niveau 1) . . . 7

4.3 Routage par instruction (niveau 2) . . . 8

5 Contexte, fils de discussion et LLM local **8**

5.1 Gestion du contexte . . . 8

5.2 Gestion des fils de discussion . . . 8

5.3 Mémoire . . . 9

5.4 Intégration du LLM local . . . 9

6 Patrons de conception utilisés et justification **9**

7 Réponses aux questions de conception **10**

8 Expériences et résultats **11**

9 Limites de la solution et améliorations possibles **11**

9.1 Limites . . . 11

9.2 Avantages et limites de l'approche multi-agents (question 14) . . . 12

9.3 Améliorations possibles . . . 12

9.4 Automatisation plus poussée : risques et garde-fous . . . 13

Contributions of group members **13**

1 Problème étudié et objectifs

1.1 Problème étudié

Un enseignant-chercheur reçoit des messages de natures très différentes : questions d'étudiants, demandes de collègues, courriers administratifs, échanges liés à un projet de recherche, messages de partenaires extérieurs. Une réponse pertinente ne dépend pas seulement du dernier message reçu, mais aussi de l'identité et du rôle de l'expéditeur, du fil de discussion, des échanges précédents, des documents de référence et des préférences de l'utilisateur.

Ces messages contiennent des informations personnelles, administratives ou scientifiques non publiées. La contrainte centrale du projet est donc que *toute génération de texte repose sur un modèle de langage exécuté localement*, afin de ne pas transmettre ces données à un service distant. Les modèles utilisables sur un ordinateur personnel sont toutefois moins capables que les très grands modèles distants.

L'hypothèse étudiée est la suivante :

> *Une organisation multi-agents composée d'agents spécialisés permet-elle d'utiliser efficacement des LLM locaux de taille raisonnable pour produire des réponses adaptées à différents contextes professionnels ?*

Plutôt que de demander à un unique modèle de traiter correctement toutes les situations, nous répartissons le travail entre des agents dotés chacun d'un rôle, d'un prompt système et d'un contexte propres, et entre des étapes courtes dont chacune est contrôlable (classer, chercher, lire, rédiger, vérifier).

1.2 Objectifs du système

Les objectifs, directement issus de l'énoncé, sont les suivants :

- **Assister, pas envoyer.** Le système propose un brouillon ; l'utilisateur garde le contrôle final. Aucun composant de l'assistant ne sait envoyer un e-mail : les brouillons sont enregistrés, jamais émis.
- **Router** chaque message vers l'agent le plus adapté et savoir se replier quand aucun ne convient.
- **Spécialiser** les agents par prompt système et par contexte documentaire, modifiables sans toucher au code Java lorsque c'est possible.
- **Gérer le contexte et les fils de discussion** en limitant la quantité d'information envoyée au LLM.
- **Rester indépendant du modèle** : remplacer un LLM local par un autre sans réécrire le framework.
- **Être extensible et mesurable** : ajouter un agent, une stratégie de routage ou une mémoire facilement, et comparer des configurations sur un jeu de scénarios.

Nous avons en outre retenu trois exigences que l'énoncé ne détaille pas mais qui découlent du contexte confidentiel : *(i)* un message est une donnée et non un ordre (défense contre l'injection de consignes), *(ii)* les brouillons ne partent qu'après validation explicite, *(iii)* chaque décision doit pouvoir être retracée.

2 Organisation multi-agents retenue

Le dépôt réunit deux réalisations complémentaires, développées en parallèle puis fusionnées. Nous les présentons comme deux *niveaux* de la même organisation.

2.1 Niveau 1 : routage par catégorie (module `email-multiagent`)

Ce niveau reprend le scénario minimal de l'énoncé : *e-mail reçu* → *agent de routage* → *agent spécialisé* → *LLM local* → *agent de routage* → *utilisateur*. Les agents sont :

`SecurityAgent` premier filtre : détecte les injections de consignes et les demandes de données sensibles ; un message suspect est mis en quarantaine avant qu'aucun autre agent ne le voie.

`RouterAgent` coordinateur. Il ne rédige jamais : il classe le message (`STUDENT`, `TEACHER`, `ADMINISTRATION`, `RESEARCH`, `GENERIC`, `SPAM`, `NO_REPLY`) avec un niveau de confiance, puis délègue.

Agents spécialisés `StudentAgent`, `TeacherAgent`, `AdministrationAgent`, `ResearchAgent` (dédié au projet de recherche) et `GenericAgent` (repli). Chacun possède son prompt système et son contexte documentaire.

`SummaryAgent` résume le message, décide s'il appelle une réponse et pré-génère un brouillon.

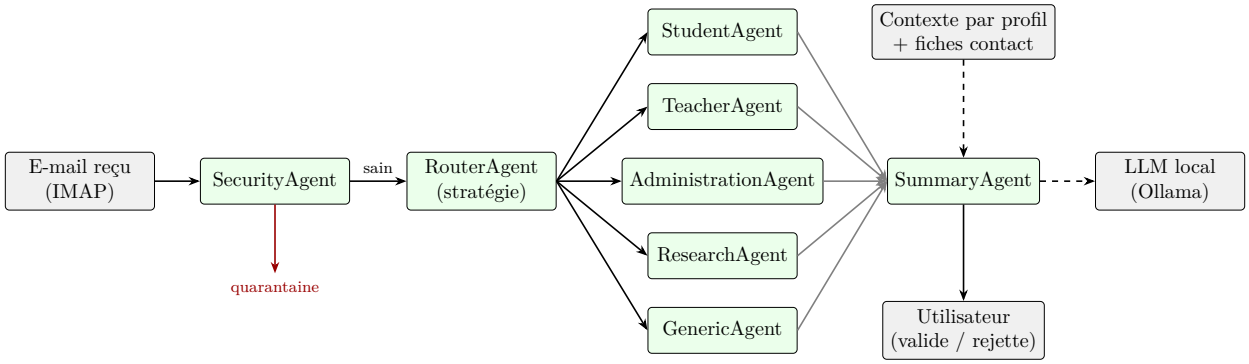


Figure 1 – Niveau 1 : organisation des agents de routage et de spécialisation.

2.2 Niveau 2 : planificateur et flux de travail (module `assistant-core`)

Ce niveau traite une *instruction de l'utilisateur* (« Réponds à Ada pour confirmer la réunion », « Résume le fil sur la journée d'étude ») plutôt qu'un message entrant. Un planificateur la décompose, trouve les messages utiles dans la boîte aux lettres, en extrait les faits et construit un *brief* que consomme un rédacteur ou un résumeur. Les rôles sont :

Planner découpe la demande en instructions atomiques, en détermine le type (`REPLY`, `COMPOSE`, `SUMMARIZE`, `CHAT`, `OUTOFSCOPE`), pilote la recherche dans la boîte, sélectionne les messages, extrait les faits, construit le brief et la checklist, et pose une question à l'utilisateur quand la demande est ambiguë ou qu'un message manque.

Writer (rédacteur) produit le brouillon à partir du brief ; un brouillon invalide est refusé par une barrière d'outils (`ToolGate`).

Summarizer (résumeur) produit un résumé fidèle et plus court que ses sources.

Critic agent critique interne à chaque rôle : il confronte le résultat à la checklist, un débat (argument, jugement) décide qui a raison et une *leçon* est mémorisée.

Mailbox la boîte aux lettres, vue comme un serveur d'outils MCP (`search_mail`, `read_mail`, `read_thread`, `create_draft`) — il n'existe volontairement aucun outil d'envoi.

2.3 Pourquoi deux niveaux ? Justification et état de l'intégration

Le niveau 1 répond à la question « à quel *type d'interlocuteur* ai-je affaire ? » : c'est un routage par catégorie, utile pour un flux entrant d'e-mails. Le niveau 2 répond à « que *veut* l'utilisateur et

de quels messages a-t-on besoin ? » : c'est de la planification d'instructions. Les deux sont utiles et se complètent, mais ils ne sont pas fusionnés en un unique pipeline : dans l'application web, le planificateur est exposé à côté du routeur (page `/assistant.html`) et lit la même boîte IMAP via un pont (`PlannerBridge`). Nous en discutons les conséquences en section 9.

Cette organisation est plus riche que le minimum demandé ; conformément à l'énoncé, nous justifions chaque extension : l'agent de sécurité (la confidentialité est la contrainte centrale), l'agent critique (compenser la faiblesse des petits modèles par une vérification structurée) et la mémoire (apprendre les préférences de l'utilisateur).

3 Architecture globale

3.1 Modules et dépendances

Le projet est un dépôt Maven multi-modules. Chaque module a une responsabilité unique et les dépendances vont dans un seul sens.

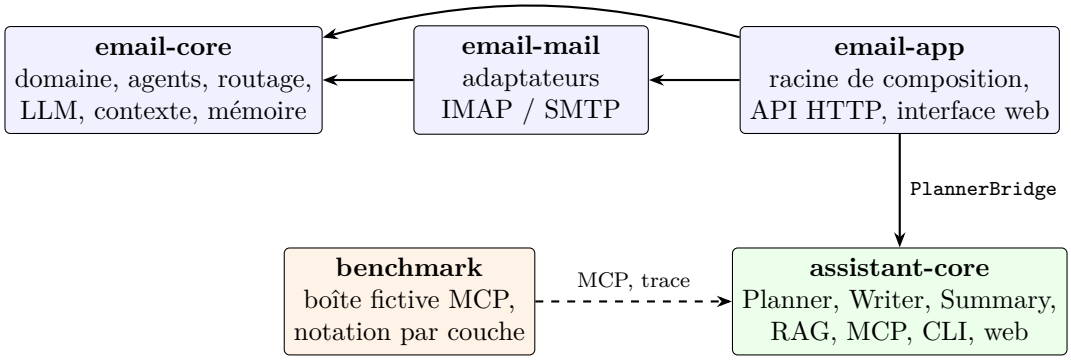


Figure 2 – Modules du dépôt. Une flèche signifie « dépend de ».

- `email-core` ne connaît ni le transport de messages ni l'interface : il ne dépend d'aucun module du projet.
- `email-mail` contient les adaptateurs IMAP et SMTP et ne connaît pas le web.
- `email-app` est la *racine de composition* : `AppContext` est le seul endroit qui choisit les classes concrètes ; tout le reste reçoit des interfaces par constructeur.
- `assistant-core` est indépendant d'`email-core` (autre paquetage `com.assistant.mail`, Java 21) ; il offre un terminal, un serveur HTTP et le point d'entrée du benchmark.
- `benchmark` évalue n'importe quel système respectant un contrat de trace ; il ne dépend pas de l'assistant.

Trois interfaces de lancement partagent le même cœur : un terminal seul (`./run.sh cli`), un chat web (`./run.sh web`) et l'application web complète (`./run.sh app`).

3.2 Classes et interfaces importantes

Table 1 – Principales classes et interfaces.

Élément	Module	Rôle
`Agent`, `AbstractAgent`	email-core	Contrat d'un agent ; `handle` fixe l'algorithme (contexte, prompt, mémoire du fil, appel du LLM), les sous-classes fournissent leur catégorie et leur prompt.

Élément	Module	Rôle
`RouterAgent`	email-core	Coordinateur : classe via une `RouterStrategy`, délègue via l'`AgentRegistry`, bascule sur `GenericAgent` si la confiance est insuffisante.
`RouterStrategy`	email-core	Port de routage : `Keyword`, `LLM`, `Hybrid`, `Feedback`.
`AgentRegistry`, `AgentFactory`	email-core	Enregistrement et création des agents par catégorie.
`LLMClient`	email-core	Port vers le modèle ; adaptateurs `OllamaClient` et `MockLLMClient`.
`ContextProvider`	email-core	Port du contexte : `FileContextProvider` et `RagContextProvider`.
`RecipientKnowledgeBase`	email-core	Fiches par contact, injectées dans les prompts, enrichies par les réponses validées.
`ConversationMemory`	email-core	Historique borné par fil.
`EmailAssistantSystem`	email-core	Façade d'orchestration (sécurité, routage, agent, mémoire).
`AppContext`, `WebServer`	email-app	Racine de composition et API HTTP.
`Planner`	assistant-core	Découpe, route l'instruction, cherche, sélectionne, lit, construit brief et checklist.
`Turn`	assistant-core	Façade d'un message utilisateur : Planner, puis rédacteur ou résumeur par brief.
`WriterWorkflow`, `SummaryWorkflow`	assistant-core	Rédaction et résumé avec marquage de checklist, débat critique et réécriture.
`ToolGate`	assistant-core	Barrière déterministe : refuse tout appel d'outil hors liste ou mal formé.
`InjectionGuard`	assistant-core	Retire des messages les ordres destinés au modèle et les signale.
`MailClientMcp`, `MailboxClient`	assistant-core	Port de la boîte aux lettres et adaptateur MCP (stdio).
`PlannerAgentClient`, `DrafterAgentClient`, `SummarizerAgentClient`	assistant-core	Ports du modèle, un par rôle ; `OllamaChat` les implémente tous.
`KnowledgeSource`, `ProfileKnowledge`, `ProfileRag`, `ContactBook`	assistant-core	Contexte par profil (RAG) et fiches contact.
`TolerantReplies`	assistant-core	Décorateur de modèle qui corrige les écarts de format.
`Assistant`	assistant-core	Façade unique pour le terminal, le web et le benchmark.

3.3 Diagramme de classes simplifié (ports et adaptateurs)

La figure 3 montre les ports du niveau 1 et leurs implémentations : le modèle, le routage et le contexte sont des interfaces, et chaque agent n'en connaît que le contrat.

3.4 Mécanismes de communication entre agents

Les agents ne partagent pas d'état global ; ils communiquent par des *objets de transfert* typés :

— **Niveau 1** : `EmailMessage` en entrée, `RoutingDecision` (catégorie, confiance, raison) et `AgentResponse` (proposition de réponse) en sortie. Le `RouterAgent` appelle l'agent choisi par l'intermédiaire de l'`AgentRegistry`.
— **Niveau 2** : `Request` → `Instruction` → `Brief` → `Draft` ou `Summary`. Le `Brief` est le

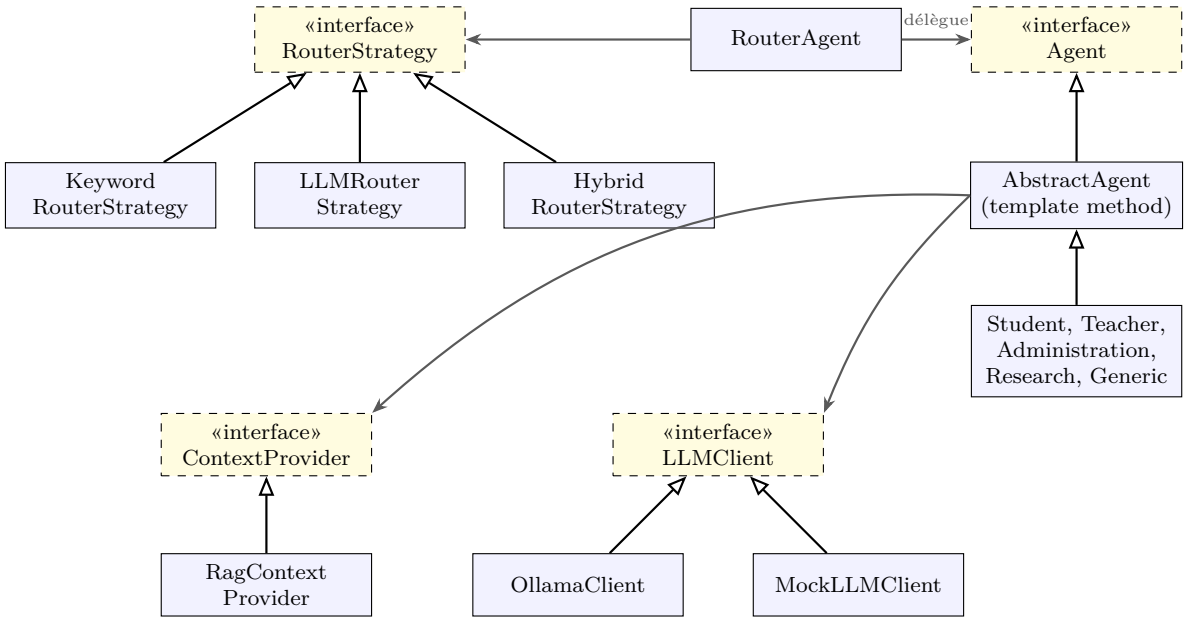


Figure 3 – Diagramme de classes simplifié du niveau 1 : le modèle, le routage et le contexte sont des ports remplaçables.

contrat entre le planificateur et le rédacteur : destinataires, langue, ton, structure, faits, consignes à ne pas suivre et lignes de connaissance. Le rédacteur ne voit jamais la demande de l'utilisateur, seulement le brief ; il ne peut donc pas être détourné par elle.

— **Outils** : la boîte aux lettres est appelée par le protocole MCP ; les appels passent par `ToolGate`, qui refuse un outil inconnu (par exemple `send`) avant tout effet.

— **Critique** : un `Dispute` (argument du critique, réponse de l'agent, vainqueur, leçon) est la forme que prend la négociation entre un agent et son critique.

4 Spécialisation des agents et stratégie de routage

4.1 Spécialisation : prompt système et contexte

Conformément à l'énoncé, la spécialisation d'un agent couvre deux dimensions.

Prompt système. Dans `email-core`, chaque agent charge son prompt depuis un fichier Markdown (`student.md`, `teacher.md`, `administration.md`, `research.md`, `generic.md`, `summarizer.md`), avec une variante française (`*_fr.md`) choisie selon la langue de l'interface. Le prompt définit le rôle, le ton, les informations à prendre en compte et à ne jamais divulguer, et la structure de la réponse. Le `PromptRepository` est un dépôt : l'agent ne connaît que le nom du prompt, jamais son emplacement. Les prompts sont des fichiers texte, donc modifiables sans toucher au code Java ; en revanche ils sont aujourd'hui *empaquetés dans les ressources* et nécessitent une recompilation (voir les limites).

Dans `assistant-core`, les prompts de chaque rôle (découpage, routage, outils, lecture, brief, critique, checklist, rédaction, résumé) sont des constantes Java regroupées par rôle (`PlannerSystemPrompt`, `DrafterSystemPrompt`, `SummarizerSystemPrompt`). Ils sont courts, de format de sortie strict (blocs `clé:\nvaleur`) et testables, mais ils ne sont pas éditables par l'utilisateur sans recompiler.

Contexte. Le contexte est propre à chaque profil : le dossier `context/` contient un sous-dossier par profil (`student/`, `teacher/`, `administration/`, `research/`) rempli de fichiers texte que l'utilisateur modifie directement. À cela s'ajoutent les fiches de contact (`recipients.json`) et l'historique du fil. Deux agents n'ont donc pas accès aux mêmes informations : le résumeur ne reçoit, par construction, que la fiche du contact et jamais les passages de profil (voir section 5.1).

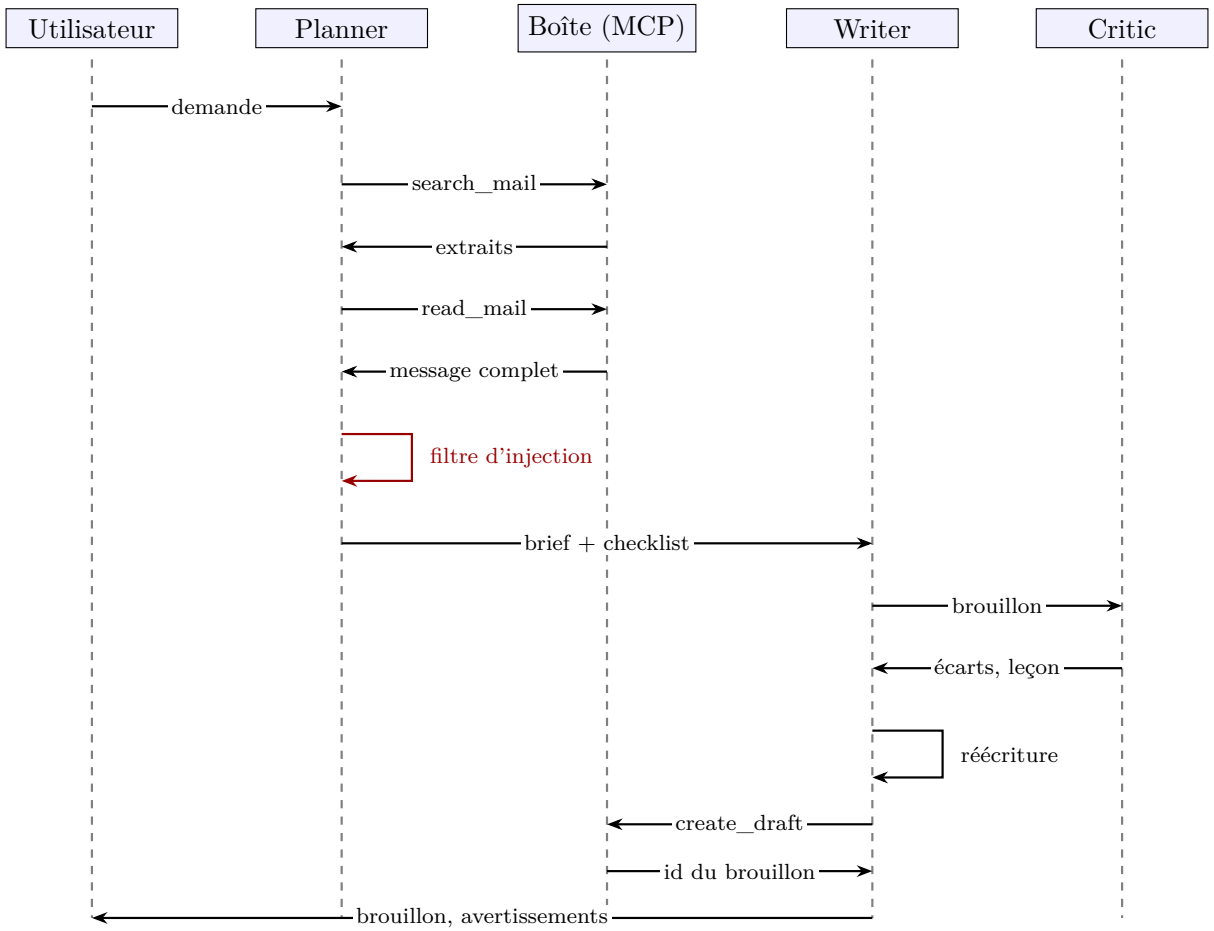


Figure 4 – Diagramme de séquence d'une demande de réponse (niveau 2). L'utilisateur valide ensuite le brouillon ; il n'est jamais envoyé automatiquement.

4.2 Stratégies de routage (niveau 1)

Le routage est un *port* (`RouterStrategy`) à quatre implémentations interchangeables, choisies par configuration (`routing.strategy`) :

Mots-clés déterministe, jamais dépendant du LLM. Il évalue le sujet et le corps contre des listes de mots (cours, projet, administration, collègue, spam, notification), chacune associée à une confiance fixe ; en cas de correspondances multiples, *la règle de plus forte confiance l'emporte*. Sous le seuil (`routing.threshold`, 0,5), il renvoie `GENERIC`.

LLM le modèle choisit une catégorie et une confiance ; en cas d'indisponibilité ou d'incertitude (sous `routing.llm.confidence`, 0,6), il retombe sur les mots-clés.

Hybride mots-clés d'abord quand la confiance est suffisante et que la catégorie n'est pas `GENERIC` ; sinon le LLM tranche. Le coût du LLM n'est payé que sur les cas difficiles.

Feedback décorateur qui abaisse la confiance d'une catégorie souvent rejetée par l'utilisateur (taux de rejet $\geq 50\,\%$: $-0{,}2$), ce qui fait basculer l'hybride vers le LLM pour les catégories fragiles.

La décision est un objet `RoutingDecision` (catégorie, confiance, *raison*) : le choix de l'agent est donc explicable et affiché à l'utilisateur. Le routage ne s'appuie pas uniquement sur l'adresse de l'expéditeur, puisqu'une même personne peut écrire dans des contextes différents.

Plusieurs agents conviennent. Pour les mots-clés, la priorité fixe des règles départage ; pour le LLM, c'est la catégorie renvoyée avec la confiance la plus haute ; le cas ambigu est mesuré dans les expériences (section 8).

Aucun agent ne convient. Sous le seuil, le `RouterAgent` délègue à `GenericAgent`, dont le prompt est volontairement prudent.

4.3 Routage par instruction (niveau 2)

Le planificateur route une *instruction*, pas un message : il en détermine le type (`REPLY`, `COMPOSE`, `SUMMARIZE`, `CHAT`, `OUTOFSCOPE`) et si elle est assez claire pour être exécutée. Trois issues ne produisent aucun brouillon : une demande hors périmètre est refusée, une salutation reçoit une réponse courte, et une demande ambiguë, sans message correspondant ou à laquelle il manque un fait donne lieu à une *question posée à l'utilisateur* (`UNCLEAR`, `NO_MAIL`, `MISSING_FACT`) plutôt qu'à une invention.

Quand le modèle ne produit pas une réponse exploitable (par exemple un refus au lieu du type), un routage de repli *par mots-clés* décide à sa place. Le modèle reste premier, la règle déterministe est un filet de sécurité.

5 Contexte, fils de discussion et LLM local

5.1 Gestion du contexte

Le contexte documentaire est *par profil*. Le flux est le suivant :

1. le destinataire du brouillon (pour une réponse : l'expéditeur du message) est identifié ;
2. sa **fiche de contact** (nom, rôle, organisation, notes, par exemple « préfère le vouvoiement ») est lue dans `recipients.json` ;
3. le **profil** est déduit du rôle donné par le planificateur, sinon du rôle de la fiche ; sans rôle connu, tous les profils sont interrogés ;
4. une recherche renvoie les **passages les plus proches** de la demande dans ce seul profil ;
5. fiche et passages sont ajoutés au brief sous forme de lignes `knowledge:`.

La recherche (`ProfileRag`) découpe chaque fichier en passages d'environ 600 caractères, par paragraphes, et classe les passages par similarité cosinus entre vecteurs d'*embeddings* locaux (modèle `all-minilm:22m` servi par Ollama, 22 millions de paramètres). Si le modèle d'embedding est absent ou en panne, la recherche retombe sur un simple recouvrement de mots normalisés (sans accents), sans interrompre la tâche. Seuls les $k = 3$ meilleurs passages sont conservés.

Éviter d'envoyer trop de contexte. Plusieurs mécanismes bornent la taille du prompt : (i) le RAG ne renvoie que $k = 3$ passages de 600 caractères au plus ; (ii) le rédacteur ne reçoit que le brief, jamais la boîte entière ni la demande brute ; (iii) un message court (moins de 1200 caractères) voyage entier comme fait, un long message est réduit à des faits atomiques extraits par le modèle ; (iv) l'historique d'un fil est borné par `memory.max.entries` ; (v) la mémoire à long terme n'injecte qu'un nombre limité de règles (≤ 10) par rôle.

Isolation. Les contextes ne se mélangent pas : un résumé n'est jamais « pollué » par des passages de profil (un essai sans cette règle a produit un résumé des procédures administratives au lieu des mails, voir section 8), et une défaillance de la source de connaissances ne bloque jamais la tâche.

5.2 Gestion des fils de discussion

— **Niveau 1** : `ConversationMemory` conserve, par identifiant de fil, les derniers échanges (fenêtre bornée) ; cet historique est envoyé à l'agent avec le message courant.
— **Niveau 2** : quand la demande parle d'un « fil », d'une « conversation » ou d'un « échange », le planificateur lit *tous les messages du fil* (`read_thread`), du plus ancien au plus récent,

et non le seul message trouvé par la recherche. Chaque résultat de recherche n'est qu'un extrait : il est ouvert en entier (`read_mail`) avant l'extraction des faits.
— **Réponses déjà générées ou envoyées** : les réponses validées par l'utilisateur alimentent la fiche du contact (`recordReply`) ; les brouillons créés sont tracés.

5.3 Mémoire

Deux niveaux : la *mémoire courte* (le fil en cours et les litiges de la tâche) et la *mémoire longue* (`LongTermMemory`), faite de règles apprises (`MemoryRule`) à partir des débats entre un agent et son critique : quand le critique a raison, la leçon est mémorisée et réinjectée dans les prompts suivants du même rôle, un point d'extension (`LessonVeto`) permettant à l'utilisateur de refuser une leçon, non câblé dans les interfaces actuelles. Cette mémoire longue vit actuellement en mémoire vive (voir les limites).

5.4 Intégration du LLM local

— **Moteur** : Ollama, par son API HTTP locale (`/api/chat`, `/api/embed`), en mode non streaming, température 0 et graine fixe pour des résultats reproductibles. Aucune donnée ne quitte la machine.
— **Indépendance au modèle** : le modèle n'est connu que par des *ports* (`LLMClient` au niveau 1 ; un port par rôle au niveau 2). Changer de modèle est un paramètre (`-model`, `ASSISTANT_MODEL` ou `llm.model`) ; un modèle absent est remplacé par le meilleur modèle installé qui suit bien les consignes, et le système le signale.
— **Prompts courts et formats stricts** : chaque étape est une petite tâche à sortie structurée (type, requête, identifiants, faits), plus adaptée à un modèle de 4 milliards de paramètres qu'un prompt unique et long.
— **Robustesse aux petits modèles** : un décorateur (`TolerantReplies`) corrige les écarts de *format* courants (appel d'outil écrit `search_mail("Ada")`, champs sur une ligne, gabarit recopié, identifiant nu) sans changer le *contenu*. Des règles déterministes complètent le modèle quand il échoue (recherche par mots distinctifs de la demande, destinataire d'une réponse, routage par mots-clés).
— **Modèles essayés** : `llama3.1:8b` (modèle de référence du benchmark) est trop lent sur notre machine (limite de 5 minutes par scénario dépassée) ; `llama3.2-abliterate:3b` ne suit pas le format demandé (il répond à la question au lieu de suivre le prompt) ; `qwen3-abliterated:4b` est le meilleur compromis testé et sert d'assistant et, pour les mesures présentées, de juge.
— **Barrières** : le modèle ne peut jamais envoyer un e-mail, ni créer un brouillon vers une adresse qu'il a inventée (`ToolGate`, correction du destinataire d'une réponse).

6 Patrons de conception utilisés et justification

Nous n'avons pas cherché à multiplier les patrons : chacun répond à un besoin concret de l'énoncé (remplacer le modèle, ajouter une stratégie de routage, un agent, une source de contexte).

Nous n'avons *pas* utilisé Observer, Mediator, Command ni Chain of Responsibility : aucun besoin réel ne le justifiait. L'appel d'outils passe par une barrière unique (`ToolGate`) plutôt que par une chaîne de responsabilités.

Principes orientés objet. Encapsulation : objets de transfert aux champs privés et accesseurs ; abstraction : ports pour le modèle, le courrier, le contexte et le routage ; faible couplage : `email-core` ignore le transport et l'interface ; forte cohésion : un module, une responsabilité ; séparation des

Patron	Où	Besoin auquel il répond
Strategy	`RouterStrategy` (mots-clés, LLM, hybride, feedback) ; `Embedder` ; `KnowledgeSource`	Ajouter ou échanger une stratégie de routage, de recherche ou de connaissance sans modifier ses clients ; comparer les stratégies sur le même jeu de test.
Template Method	`AbstractAgent.handle`	Tous les agents suivent le même déroulement (contexte, prompt, mémoire, LLM, réponse) ; une sous-classe ne fournit que sa catégorie et son prompt. Ajouter un agent revient à écrire une petite classe.
Factory	`AgentFactory`, `RouterFactory`, `LLMClientFactory`, `MailClientFactory`, `Assistant.create`	Isoler le choix des classes concrètes selon la configuration.
Registry	`AgentRegistry`	Retrouver l'agent d'une catégorie ; permet d'enregistrer un agent sans modifier le routeur.
Adapter	`OllamaClient`, `OllamaChat`, `ImapMailClient`, `SmtpMailSender`, `MailboxClient` (MCP)	Adapter un service extérieur (Ollama, IMAP, serveur MCP) à une interface du domaine.
Facade	`EmailAssistantSystem`, `Turn`, `Assistant`	Offrir un point d'entrée unique et simple au terminal, au web et au benchmark.
Decorator	`FeedbackRouter`, `TolerantReplies`	Ajouter un comportement (pondération par le retour utilisateur, correction du format) autour d'un composant sans le modifier.
Repository	`PromptRepository`, `RecipientKnowledgeBase`, `AuditLog`, `FeedbackStore`	Séparer l'accès aux données persistantes de leur usage.
Composition root	`AppContext`	Un seul endroit construit les objets ; le reste ne dépend que d'interfaces (faible couplage).
Pipeline / chaîne d'étapes	`Planner` puis `Writer`/`Summary`	Étapes courtes à sortie structurée, chacune vérifiable ; une étape ne connaît que le contrat de la suivante.

Table 2 – Patrons de conception et justification.

responsabilités : le routeur ne rédige pas, le rédacteur ne cherche pas, le planificateur ne rédige pas ; extensibilité et réutilisation : `email-core` ou `assistant-core` peuvent être repris seuls.

7 Réponses aux questions de conception

Cette section répond, dans l'ordre, aux quatorze questions de la section 14 de l'énoncé.

1. **Qu'est-ce qu'un agent dans notre architecture ?** Un composant doté d'un rôle, d'un prompt, d'un contexte et d'une interface étroite : au niveau 1, un objet `Agent` spécialisé par catégorie ; au niveau 2, un rôle (Planner, Writer, Summarizer, Critic) dont l'entrée est un brief ou une instruction.
2. **Quelles sont ses responsabilités ?** Une seule : le routeur classe, le spécialisé rédige pour une catégorie, le résumeur résume, le critique vérifie, la barrière d'outils contrôle.
3. **Quelle information possède-t-il ?** Son prompt, son contexte de profil (RAG), la fiche du contact et l'historique du fil pour les agents de niveau 1 ; uniquement le brief pour le rédacteur.
4. **Comment communiquent-ils ?** Par objets typés (`RoutingDecision`, `AgentResponse`, `Brief`, `Draft`) et par appels d'outils MCP, sans état partagé (section 3).

5. **Comment le routeur détermine-t-il l'agent ?** Par une stratégie interchangeable (mots-clés, LLM, hybride, pondérée par le retour utilisateur) avec une confiance et une raison.
6. **Plusieurs agents conviennent ?** La règle de plus forte confiance l'emporte (mots-clés) ou le LLM tranche (hybride) ; le cas ambigu est mesuré.
7. **Aucun agent ne convient ?** Repli sur `GenericAgent` au niveau 1 ; au niveau 2, question posée à l'utilisateur, refus d'une demande hors périmètre.
8. **Comment les contextes sont-ils représentés ?** Fichiers texte par profil, découpés en passages et indexés par embeddings ; fiches de contact JSON ; historique par fil ; règles apprises.
9. **Comment l'historique d'une conversation est-il géré ?** Mémoire bornée par fil au niveau 1 ; lecture de tous les messages du fil à la demande au niveau 2.
10. **Comment éviter d'envoyer trop de contexte ?** Recherche des $k = 3$ passages pertinents d'un seul profil, brief au lieu de la boîte entière, faits atomiques, fenêtres bornées, règles limitées à dix (section 5.1).
11. **Comment ajouter un agent sans modifier l'application ?** Écrire une sous-classe de `AbstractAgent`, l'enregistrer dans l'`AgentRegistry`, ajouter son prompt et son dossier de contexte ; le routeur et l'interface n'ont pas à changer (hors liste de catégories et mots-clés).
12. **Comment changer le LLM ?** Un paramètre de configuration ; les agents ne connaissent que `LLMClient` ou les ports de rôle.
13. **Quelles données doivent être persistantes ?** Persistantes aujourd'hui : configuration, prompts, documents de contexte, fiches de contact, journal d'audit et retours utilisateur (JSONL). À rendre persistantes : règles de la mémoire longue et brouillons (actuellement en mémoire vive).
14. **Avantages et limites de l'approche multi-agents ?** Voir la section 9 : gains de contrôle, de traçabilité et de robustesse avec de petits modèles ; coût en latence, en complexité et en étapes de vérification.

8 Expériences et résultats

À venir.

9 Limites de la solution et améliorations possibles

9.1 Limites

Organisation et architecture.

— Les deux niveaux (routage par catégorie et planificateur) *coexistent mais ne forment pas un pipeline unique* : le planificateur n'utilise pas les agents spécialisés de catégorie, et le routeur n'utilise pas le planificateur. Le pont (`PlannerBridge`) ne fait que partager la boîte aux lettres.

— `EmailCategory` est une énumération : ajouter une catégorie demande de modifier cette énumération et les mots-clés de routage, ce qui contredit en partie l'objectif « ajouter un agent sans modifier l'application ».

— Les prompts de `email-core` sont des fichiers texte empaquetés dans les ressources, ceux de `assistant-core` des constantes Java : l'utilisateur ne peut pas les modifier à l'exécution sans recompiler.

Interface. L'interface web permet de voir la boîte, la catégorie choisie par le routeur, le résumé, le brouillon éditable, d'envoyer ou de rejeter, et de consulter les contacts. En revanche, elle **ne**

permet pas de créer ou de modifier un agent, d'éditer son prompt système ni de lui associer des fichiers ; ces points de l'énoncé (section 10) ne sont pas couverts.

Données et mémoire. La mémoire longue (règles apprises) et les brouillons du niveau 2 sont en mémoire vive et perdus à l'arrêt ; l'apprentissage des préférences n'est donc pas durable.

Modèles locaux. Un modèle de 4 milliards de paramètres ne suit pas toujours le format des prompts, invente parfois un détail dans un résumé et laisse des éléments factices (« [Votre nom] »). Nous compensons par des corrections de format et des règles déterministes, mais ces filets de sécurité masquent la faiblesse du modèle plus qu'ils ne la suppriment. La latence est élevée (de l'ordre de 40 à 130 secondes par demande sur notre machine) ; le modèle de référence du benchmark (`llama3.1:8b`) dépasse la limite de 5 minutes par scénario sur ce matériel.

Sécurité. Le filtre d'injection (`InjectionGuard`) est à base de règles : il attrape les formes courantes (jetons de gabarit de conversation, « ignore les consignes précédentes », ordre de créer un brouillon vers une autre adresse), pas toutes ; il peut aussi, plus rarement, signaler à tort une phrase légitime.

Évaluation. Les mesures de la section 8 portent sur *trois* des vingt scénarios du benchmark, une exécution chacun, avec un juge de petite taille ; elles ne permettent aucune conclusion statistique. Le benchmark complet avec le juge de référence n'a pas été exécuté. La comparaison avec une architecture à agent unique, suggérée comme extension, n'a pas été réalisée. La boîte IMAP réelle n'est pas exercée dans les expériences rapportées.

9.2 Avantages et limites de l'approche multi-agents (question 14)

Avantages observés. (i) *Contrôle* : chaque étape est courte, à sortie structurée et vérifiable, ce qui permet d'insérer des barrières déterministes entre le modèle et l'effet (aucun outil d'envoi, destinataire contrôlé) ; (ii) *traçabilité* : la trace indique l'étape fautive ; (iii) *spécialisation* : prompts et contextes séparés, sans mélange d'informations entre profils ; (iv) *robustesse* : une étape qui échoue est rattrapée localement sans refaire toute la tâche ; (v) *testabilité* : chaque rôle se teste avec un modèle simulé.

Limites. Les étapes multiplient les appels au modèle (donc la latence), la coordination ajoute de la complexité, et la qualité du résultat reste bornée par le modèle le plus faible de la chaîne. Une architecture simple à agent unique serait plus rapide ; l'intérêt du multi-agents ici est la fiabilité et le contrôle, pas la vitesse.

9.3 Améliorations possibles

- Fusionner les deux niveaux : le planificateur choisirait un agent spécialisé pour la rédaction.
- Interface de création et d'édition d'agents, de prompts et de contextes ; répertoire de prompts externe (`prompts.dir`) lu à chaud.
- Rendre la mémoire longue et les brouillons persistants, et brancher le droit de veto de l'utilisateur.
- Remplacer l'énumération de catégories par des profils chargés depuis des fichiers de configuration.
- Détection d'injection complétée par un classifieur ; tests d'attaque plus nombreux.
- Exécuter les vingt scénarios avec le juge de référence, répéter les mesures (`-repeat`) et comparer agent unique et multi-agents.

— Recherche de contexte améliorée (reclassement, métadonnées de date), et estimation de la confiance du routeur calibrée sur les retours.

9.4 Automatisation plus poussée : risques et garde-fous

Un envoi automatique est possible à terme, mais seulement avec des garde-fous : seuil de confiance minimal, liste blanche de destinataires, validation humaine obligatoire dès qu'un message est en quarantaine ou contient un ordre suspect, journal d'audit immuable, limitation du débit, mode « simulation » préalable, et actions réversibles (brouillon plutôt qu'envoi). Le choix actuel (aucun outil d'envoi dans l'assistant) est la forme la plus sûre de ces garde-fous.

Contributions of group members

Cette section doit refléter le travail réellement effectué. Les noms, numéros d'étudiant et contributions individuelles sont à **compléter et valider par le groupe** : nous ne les inventons pas.

Étudiant	Numéro étudiant	Contribution
Valentin **[À COMPLÉTER : NOM]**	**[À COMPLÉTER : n°]**	**[À COMPLÉTER : parties du projet et principaux développements]**
Kyllian **[À COMPLÉTER : NOM]**	**[À COMPLÉTER : n°]**	**[À COMPLÉTER : parties du projet et principaux développements]**
Jade **[À COMPLÉTER : NOM]**	**[À COMPLÉTER : n°]**	**[À COMPLÉTER : parties du projet et principaux développements]**
Théolien BIERHOFF	**[À COMPLÉTER : n°]**	**[À COMPLÉTER : parties du projet et principaux développements]**

Aide à la complétion : historique du dépôt. L'historique git de la branche fournit des indices, sans remplacer l'avis du groupe. Le module `email-multiagent` a été importé depuis un dossier sans historique git : ses auteurs n'y apparaissent pas.

Auteur git	Commits	Sujets des commits
Bierhoff Theolien	12	dépôt initial (workflows et données), objets du domaine de chaque composant, workflow du planificateur, agent critique-planificateur, workflow déterministe
Diekope	3	fusion des deux projets, module `assistant-core` exécutable (terminal, web, benchmark), RAG par profil et fiches contact, filtre d'injection, correctifs pilotés par le benchmark, rapport

[À COMPLÉTER : le groupe indique qui a écrit l'agent de routage, les agents spécialisés, l'interface web, l'intégration IMAP/SMTP, le RAG d'`email-multiagent`, le benchmark et les scénarios]